

NATIONAL ECONOMICS UNIVERSITY – SCHOOL OF TECHNOLOGY

FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

COURSE HANDBOOK & SELF-STUDY GUIDE

DSAI1003: FUNDAMENTAL PROGRAMMING CONCEPTS IN PYTHON

An Essential Foundation for Data Science & AI

Instructor: Dr. Vu Duc Minh

Program: Data Science in Finance & E-commerce (DSFE)

Workload: 3 Credits (45h LT + 22.5h Lab)

Decision: 975/QĐ-ĐHKTQĐ (30/03/2024)

Website: <https://vudmn.github.io/Fundamental-of-Programming-Concepts-in-Python/>

Course Presentation Agenda

Part 1: Course Overview (Syllabus)

- ▶ Course description & pedagogical approach
- ▶ Learning objectives (COs G1 – G7)
- ▶ International textbooks & grading scheme
- ▶ 15-week detailed teaching schedule

Part 2: Effective Self-Study Methods

- ▶ 4 fatal pitfalls & core principles
- ▶ The 9-step active study cycle
- ▶ Predict → Run → Explain & Teach-Back
- ▶ 8-step debugging & Personal Error Log

Part 3: Leveraging AI for Breakthroughs

- ▶ Positioning: AI Personal Tutor & Coach
- ▶ 4-part structured prompt framework
- ▶ 5T Principle & 3-level hint system
- ▶ Actionable Prompt Toolkit & Verification

Part 4: Week 1 Action Plan

- ▶ Environment setup (VS Code, Colab)
- ▶ Explore GitHub course resources
- ▶ Initialize personal Learning Log
- ▶ Commit to active learning culture

PART 1

COURSE OVERVIEW (SYLLABUS)

Course Description, Objectives, Textbooks, Assessment & 15-Week Roadmap

1. Course Description & Pedagogical Approach

Curricular Position & Audience

- ▶ **Course Code:** DSAI1003 (3 Credits).
- ▶ **Target Audience:** Undergraduate students in *Data Science in Finance & E-commerce (DSFE)*.
- ▶ **Curricular Nature:** **Compulsory core foundation** establishing algorithmic mindset for advanced DS/AI studies.

Pedagogical Philosophy

- ▶ **No prior experience required:** Only standard high school algebra and logical reasoning needed.
- ▶ **Practical real-world focus:** Case studies in business, finance, retail analytics, and e-commerce.
- ▶ **Structured progression:** Algorithmic logic → Syntax → Built-in collections → OOP.

Learning Commitment: Students will gain the competence to independently design and implement robust programs solving business analytics tasks, fully prepared for Machine Learning and AI courses.

2. Course Objectives (Course Learning Outcomes)

Algorithmic Logic & Core Syntax

- ▶ **G1 (Algorithmic Thinking):** Express algorithms via pseudocode and flowcharts; implement console/GUI apps.
- ▶ **G2 (Python Foundations):** Master data types, expressions, decision branches, loops, and modular functions.
- ▶ **G3 (Data Collections):** Manage data efficiently using Lists, Tuples, Sets, and Dictionaries.

Practical Applications & DSFE Focus

- ▶ **G4 (File I/O & Exceptions):** Safe file operations (text/binary) and robust exception handling.
- ▶ **G5 (Object-Oriented Programming):** Model domain entities with Classes, Objects, and Inheritance.
- ▶ **G6 (Modern Developer Tools):** Proficient use of VS Code, Git, Google Colab, and AI assistants.
- ▶ **G7 (Applied Real-World Projects):** Solve end-to-end data processing tasks in business and finance.

3. International Textbooks & Grading Scheme

3 Standard International Textbooks

1. **[1] Python for Everyone** (3rd Ed., 2019)
Cay Horstmann, Rance Necaise (Wiley)
→ **Main textbook** for syntax & algorithmic design.
2. **[2] Intro to Python for CS and Data Science** (2020)
Paul Deitel, Harvey Deitel (Pearson)
→ **Main textbook** for data processing & AI.
3. **[3] The Python Workbook** (2nd Ed., 2019)
Ben Stephenson (Springer)
→ Self-study exercises with complete solutions.

Assessment Components (3 Grades)

Component	Scope & Criteria	Weight
Homework (HW)	Theory & coding (LMS, bi-weekly)	30%
Midterm Exam	Week 7 exam (Lessons 1 – 6)	30%
Final Exam	Computer lab practical test (Week 15)	40%

Academic Regulations

- ▶ **Attendance:** Minimum **80%** class attendance required.
- ▶ **Passing grade:** Final composite grade ≥ 5.0 / 10.0.

4. 15-Week Teaching Plan – Phase 1 (Weeks 1 – 7)

Week	Lesson & Topic	Core Knowledge Focus	Assessment
1	Lesson 1: Intro & Algorithms	Computers, Python interpreter, pseudocode, flowcharts	Environment setup
2	Lesson 2: Software Process & Types	Numbers, strings, assignments, variables, modules	Homework #1
3	Lesson 3: Decision Structures	Branching if, if-else, if-elif-else, Boolean	Due: HW #1
4	Lesson 4: Loop Structures	Loops while, for, nested loops, strings	Homework #2
5	Lesson 5: Function Design	Syntax def, parameters, return, scope	Due: HW #2
6	Lesson 6: Lists & Tuples	1D/2D arrays, slicing, built-in methods	Homework #3
7	Midterm Examination	Synthesis & assessment of Lessons 1 – 6	Midterm (30%)

5. 15-Week Teaching Plan – Phase 2 (Weeks 8 – 15)

Week	Lesson & Topic	Core Knowledge Focus	Assessment
8–9	Lesson 7: File I/O & Exceptions	Text file reading/writing, <code>sys.argv</code> , <code>try-except</code>	Homework #4
10	Lesson 8: Sets & Dictionaries	Key-value mappings, hash tables, nested JSON records	E-commerce case study
11–12	Lesson 9: Objects & Classes (OOP 1)	OOP mindset, <code>class</code> , <code>__init__</code> , methods	Mini-lab OOP
13–14	Lesson 10: Inheritance & Polymorphism	Inheritance hierarchy, <code>super()</code> , method overriding	Final exam review
15	Course Review & Exam Prep	Comprehensive synthesis of all course topics	Mock lab test

PART 2

EFFECTIVE PYTHON SELF-STUDY METHODS

Overcoming Pitfalls, 9-Step Study Cycle, Active Learning & Systematic Debugging

6. The Nature of Programming & 4 Fatal Pitfalls

"Learn programming by reading less, practicing more, and thinking before looking at solutions."

Pitfall 1: Reading without running code

Skimming sample code and assuming understanding. When the screen is off, cannot reproduce a single line.

Pitfall 2: Blind copy-pasting

Copying code from the web or AI; code executes, but the learner cannot explain why specific statements exist.

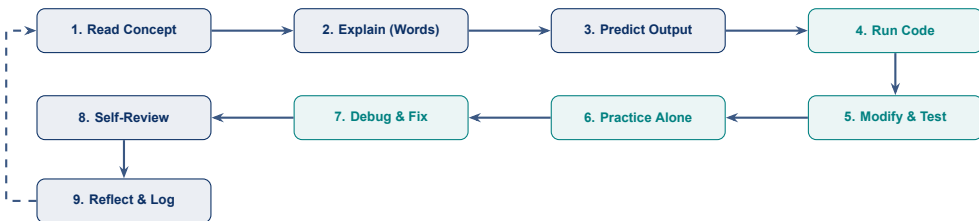
Pitfall 3: Checking solutions prematurely

Opening solutions immediately when stuck. Bypasses the productive cognitive struggle where problem-solving develops.

Pitfall 4: Cramming too many topics

Rushing from variables to OOP in 2 days without allowing time for concepts to consolidate through practice.

7. The 9-Step Active Programming Study Cycle



The 70/30 Golden Rule of Self-Study:

Allocate at most **30% of study time** to reading theory; dedicate over **70% of time** directly to hands-on coding: typing, observing runtime errors, analyzing tracebacks, and solving unassisted problems!

8. Breakthrough Active Learning Techniques

Predict → Run → Explain

Before clicking Run on the IDE:

- ▶ Write down exact expected terminal output.
- ▶ Predict how memory and variables change state.
- ▶ If output differs → That is where maximum learning occurs!

Teach-Back Technique

Explain concepts to a beginner in simple terms. Saying *"It works because it has to"* proves you do not truly understand the mechanism!

Active Recall (Self-Testing)

Do not repeatedly reread slides!

- ▶ Close notes and answer diagnostic checks:
 - ▶ What type does `input()` return?
 - ▶ How do you cast to an integer?
 - ▶ Why does `string + integer` cause an error?

Spaced Repetition Schedule

Review on an expanding interval:

Day 1 → Day 3 → Day 7 → Day 14

Converts short-term memory into lasting intuition.

9. The 5-Level Learning Ladder

Level 5 – Mini Project: Build a console order management system or transaction log analyzer.

Level 4 – Real-World Problems: Compute progressive electricity tariffs, tax brackets, compound interest.

Level 3 – Combine Concepts: Concurrently apply a `for` loop and an `if-else` branch to filter records.

Level 2 – Reconstruct from Description: Cover sample solutions, read prompts, code completely from scratch.

Level 1 – Modify Existing Code: Change variable values, convert integers to floats, observe output changes.

Never leap directly from basic examples to complex projects without ascending each intermediate ladder rung!

10. Debugging Skills & Standard Error Classification

Standard 8-Step Debugging Protocol

1. Read the traceback from the bottom line upward.
2. Pinpoint the exact file and line number.
3. Identify the error type (`TypeError`, `IndexError`...).
4. Insert `print()` statements to inspect variables.
5. Use `type()` to verify runtime data types.
6. Compare observed output against expectations.
7. Test against minimal, simplified input sets.
8. **Change only 1 element** between test runs!

Distinguishing 3 Error Categories

- ▶ **Syntax Error:** Violating language grammar (missing `:`, unmatched parentheses). Caught immediately upon loading.
- ▶ **Runtime Error:** Valid syntax that crashes during execution (division by zero, invalid type casting).
- ▶ **Logic Error:** Program runs without errors but **produces incorrect calculations!** (Most dangerous).

11. Personal Learning Log & Error Tracking (Error Log)

Transforming Mistakes into Intellectual Assets:

Documenting encountered errors systematically converts painful bugs into invaluable personal mastery that prevents exam errors.

Error Message	Root Cause	Correction Strategy	Clear?
<code>TypeError: can only concatenate str to str</code>	<code>input()</code> always returns a string (<code>str</code>), which cannot be added to numbers.	Explicitly cast: <code>int(input())</code> or <code>float(input())</code> .	✓
<code>IndexError: list index out of range</code>	Valid indices range from 0 to $N - 1$; accessing index N causes out-of-bounds error.	Fix iteration boundaries: <code>range(len(lst))</code> .	✓
Infinite loop (<code>while</code>)	Forgot to increment step variable in loop body.	Add statement: <code>i += 1</code> .	✓

Maintain your `Learning_Log.md` file after each study session to monitor weekly growth!

PART 3

STRATEGIC AI (CHATGPT) ASSISTED LEARNING

AI Tutor, 4-Part Prompt, 5T Principle, Prompt Toolkit & Verification

12. Positioning AI Accurately in Programming Education

WARNING: Harmful Over-Reliance

Flawed Workflow:

Prompt → Delegate to AI → Copy code → Paralysis!

- ▶ Zero understanding of why statements are chosen.
- ▶ Completely helpless in proctored exam rooms without AI.
- ▶ Analytical problem-solving ability atrophies over time.

RECOMMENDED: AI as Personal Coach

Best-Practice Workflow:

Think → Draft code → Request hints → Code!

- ▶ Ask AI to **explain** difficult concepts in plain terms.
- ▶ Request **incremental hints** without full solutions.
- ▶ Ask AI to **quiz your understanding** interactively.
- ▶ Request **code reviews** on your own hand-written code.

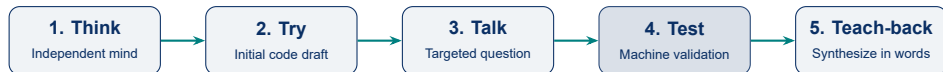
13. The 4-Part Structured Prompt Framework

Component	Guiding Question	Practical Concrete Example
1. Context	Who are you, what level?	<i>"I am a first-year student beginning foundational Python programming..."</i>
2. Problem	Where are you stuck?	<i>"I struggle to differentiate between variables and assigned values..."</i>
3. Task	What should AI do?	<i>"Explain with an everyday analogy and a Python snippet under 8 lines..."</i>
4. Constraint	Boundaries on the response?	<i>"Do not use functions or OOP; ask me 2 check questions without answers yet."</i>

The Key Difference:

Compared to blunt queries like *"What is a variable in Python?"*, structured prompts allow AI to tailor explanations precisely to your current knowledge level!

14. The 5T Principle for AI-Assisted Study



- ▶ **Think:** Formulate Inputs, Outputs, and algorithms before opening AI chat.
- ▶ **Try:** Type your first draft independently, even with bugs.
- ▶ **Talk:** Provide existing code and ask about the exact bottleneck.
- ▶ **Test:** Always run code on IDE; test edge cases (0, negatives, empty lists).
- ▶ **Teach-back:** Close chat window and explain the solution in your own words.

15. The Three-Level Hint System

When tackling difficult exercises, request tiered hints – Never ask for complete solutions!

Level 1: Idea Hint

(Conceptual Direction)

Analyzes the problem and outlines mathematical/algorithmic logic.

→ **Strictly zero lines of code provided!**

Level 2: Structure Hint

(Architectural Clues)

Suggests key statements: `for` loops, nested `if`, modulo operator `%...`

→ **Shapes the logic scaffold.**

Level 3: Scaffold Hint

(Skeleton Template)

Provides template code with blank placeholders (`# TODO`) to implement.

→ **Preserves intellectual ownership.**

Problem-Solving Strategy: Strive to solve problems using Level 1 hints. Only unlock Levels 2 and 3 after 15 minutes of unassisted effort!

16. Actionable Prompt Toolkit (1): Concepts & Hints

New Concept (Concept Prompt)

I am a beginner learning Python. Please explain the concept of [Recursive Functions].

Requirements:

1. Intuitive explanation with 1 real-world analogy.
2. Short code example (under 10 lines).
3. 1 common pitfall beginners frequently make.
4. 2 check questions without answers yet.

Exercise Hint (Socratic Hint)

I am solving this exercise: "[PROBLEM DESCRIPTION]"

My current code attempt:

[PASTE YOUR CODE HERE]

I am blocked at step [DESCRIBE BOTTLENECK].

Requirement: ABSOLUTELY DO NOT write code for me. Give me a LEVEL 1 HINT regarding algorithmic ideas so I can think through it.

17. Actionable Prompt Toolkit (2): Debug & Code Review

Debugging Support (Root Cause)

I wrote Python code to achieve: [CODE OBJECTIVE]

Current code: [PASTE CODE]

Encountered error: [PASTE TRACEBACK]

Requirements:

1. Explain the root cause of this error.
2. Point out the problematic line(s).
3. Ask 1 guiding question so I can fix it myself; DO NOT rewrite my code.

Code Review & Optimization

I completed this exercise and code executes correctly:
[PASTE WORKING CODE]

Act as an experienced Python Instructor:

1. Evaluate variable naming, readability, logic structure.
2. Does the code handle edge cases safely?
3. Suggest 1 concrete Pythonic optimization and explain why.

18. Actionable Prompt Toolkit (3): Self-Quiz Generation

Generating Practice Quizzes (Self-Quiz Prompt)

I just finished studying: **[Lists, Tuples, Slicing, and for Loops]**.

Create a 6-question diagnostic quiz for me:

- ▶ **2 Multiple-Choice:** 4 options A, B, C, D (exactly 1 correct answer).
- ▶ **2 Predict the Output:** Code snippet 3–5 lines using foundational features only.
- ▶ **2 Find the Bug:** Code snippet containing exactly 1 bug to identify and fix.

Constraint: DO NOT show answers upfront. Wait for my submitted answers before grading and providing explanations!

Advanced Pro-Tip: Quizzes from Past Mistakes

Copy past error entries from your `learning_log.md` into the prompt: *"Create 5 quiz questions specifically targeting these past bugs to test whether I have truly overcome them!"*

19. 7-Point AI-Generated Code Verification Checklist

- ☐ **1. Curriculum Match:** Does the code avoid external libraries or advanced concepts beyond syllabus?
- ☐ **2. Practical Run:** Have you run the code in IDE without compilation/runtime errors?
- ☐ **3. Exact Spec Match:** Does output match problem requirements down to exact formatting?
- ☐ **4. Boundary Testing:** Does code handle edge cases (0, negatives, empty lists)?
- ☐ **5. Standard Terms:** Are terms aligned with official textbook (*Python for Everyone*)?
- ☐ **6. Line-by-Line Explanation:** Can you point to every line and explain why it exists?
- ☐ **7. Independent Reproduction:** Can you rewrite this code in 10 minutes without AI?

The Inviolable Golden Rule:
"NEVER SUBMIT OR PRESERVE SOURCE CODE
THAT YOU CANNOT THOROUGHLY EXPLAIN YOURSELF!"

20. Immediate Action Plan for Week 1

1. Environment Setup

- ▶ Install **Python 3.11+** from python.org.
- ▶ Install **VS Code** editor with official Python extension pack.
- ▶ Set up a **Google Colab** account in your browser for instant coding.

2. Resource Exploration

- ▶ Bookmark course GitHub repo: github.com/vudmnv/Python.
- ▶ Read the detailed syllabus ([syllabus-en.md](#)).
- ▶ Read Chapter 1 of textbook *Python for Everyone*.

3. Self-Study Log

- ▶ Create study folder `DSAI1003_Python/` on machine.
- ▶ Create file `learning_log.md` to log weekly progress.
- ▶ Save the Prompt Toolkit to interact with AI ethically and smartly.

WISHING ALL DATA SCIENCE & AI STUDENTS
MASTERY OF COMPUTATIONAL THINKING & BREAKTHROUGH CAPABILITIES WITH
AI!

THANK YOU!

FOR YOUR ATTENTION & ENGAGEMENT

Dr. Vu Duc Minh

Faculty of Data Science and Artificial Intelligence – School of Technology
National Economics University (NEU)

Website:

<https://vudmn.github.io/Fundamental-of-Programming-Concepts-in-Python/>